

IMP

Mikroprocesorové a vestavěné systémy

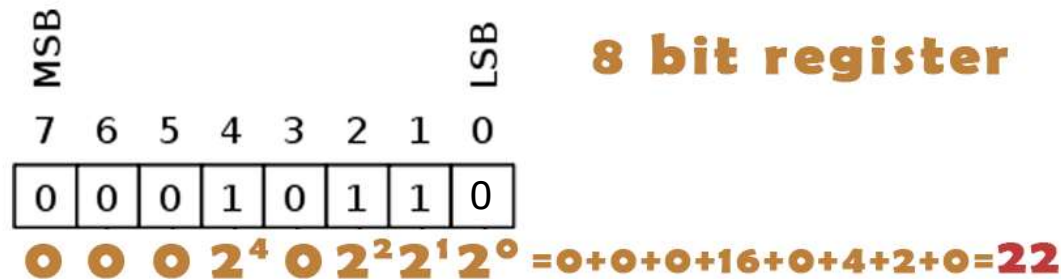
Organizace

- **Přednášky** vždy ve čtvrtek od 12:00 v D105.
- „**Demo**“ **cvičení** – jen některé týdny po přednášce (celkem 6).
- Za body:
 - **Laboratorní cvičení** – od 5. týdne v L306, celkem 4 úlohy, 1x za 14 dní, až **16b**.
 - **Projekt** – samostatné řešení, výběr zadání od 3. týdne, až **14b** (pro **zápočet** min. 5).
 - **Půlsemestrálka** v termínu přednášky dne 7. 11. 2024, až **19b**.
- **Písemná zkouška** za **51b**, minimum 20.

Smysl předmětu

- ukázat, jak to vypadá **tam, kde se „dotýká“ hardware a software**
 - jak vlastně dosáhnu toho, že psaním kódu se něco stane ve fyzickém světě, že z nehmotných příkazů je najednou něco, co „pohne hmotou“?

To se nejlépe ukazuje na „embedded“, ale platí to obecně.



Smysl předmětu

- Ukázat, **co je to „embedded“**, čím je specifické a čím je stejné jako jiné počítačové systémy.
- Ukázat, že vlastně **i čistý programátor má moc ovlivňovat fyzický svět** a když si to neuvědomuje, že **může napáchat i velké škody**, přispět k poškozování životního prostředí.
- Ukázat, že každý si dneska může **snadno „naprototypovat“** nějaký **jednoduchý „gadget“** (pokud chápe základní principy). Bylo by smutné, kdyby absolventi informatické VŠ nevěděli alespoň trochu víc.

Vestavné systémy

Mikroprocesorové a vestavěné systémy (IMP)

Richard Růžička

Fakulta informačních technologií VUT v Brně

Odkud se vzala potřeba vestavěných systémů?

Zpočátku byly systémy realizovány **mechanicky**,

Později pokrok umožnil realizovat systémy **elektricky** (elektronicky),

Dnes bychom nejraději realizovali systémy **jako software**.

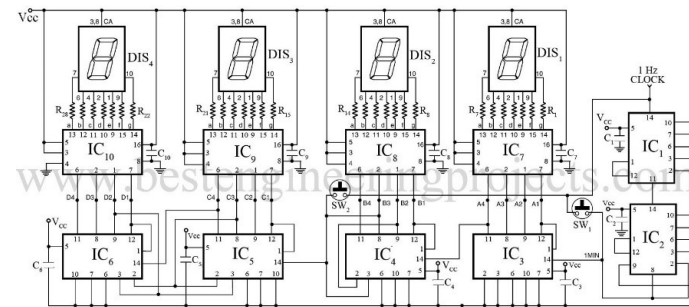
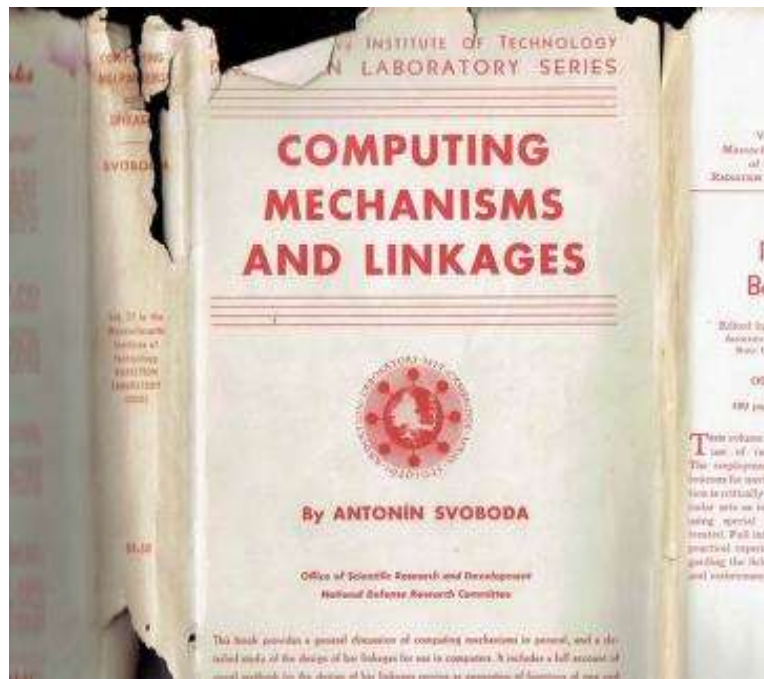


Figure 1: 24 Hour Digital Clock Circuit

```
if (milliseconds >= 1000) {  
    seconds++;  
    milliseconds=0;  
}  
  
if (seconds >= 60) {  
    minutes++;  
    seconds = 0;  
}  
  
if (minutes >= 60) {  
    hours++;  
    minutes = 0;  
}  
  
if (hours >= 24) {  
    hours = 0;  
}
```

Zpočátku byly systémy realizovány
mechanicky.



← Kniha A. Svobody z roku 1948

Antonín Svoboda, matematik z ČVUT, v ní popisuje, jak realizovat mechanicky výpočetní systémy.

V letech 1936 – 1938 navrhl mechanický počítač pro řízení protiletadlové palby pro armádu ČSR.

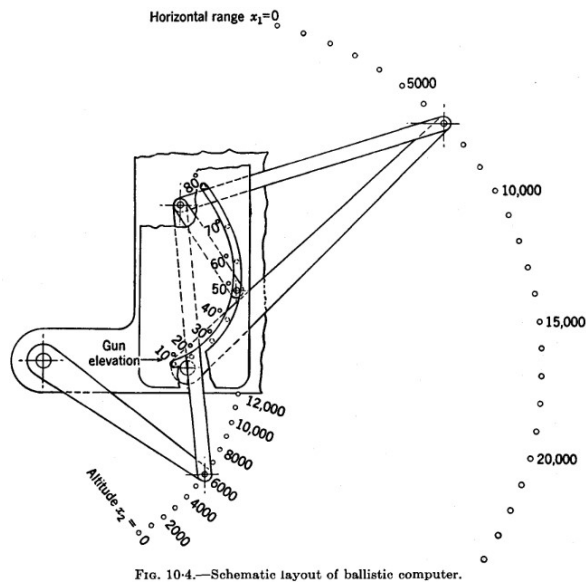
Během války pracoval na tématu na MIT v USA a navrhl radarem řízený protiletadlový systém pro US NAVY (MARK 56).

1950 navrhoval první čs. elektronický počítač SAPO

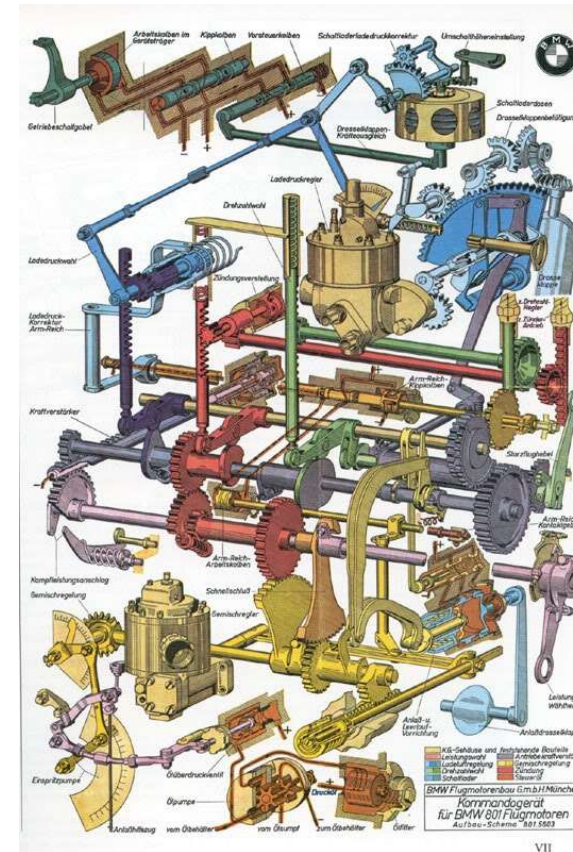
1956 elektronkový počítač EPOS

1964 odchází opět do USA a stává se profesorem na UCLA

Mechanické systémy



Jednoduché schéma balistického počítače ze Svobodovy knihy.



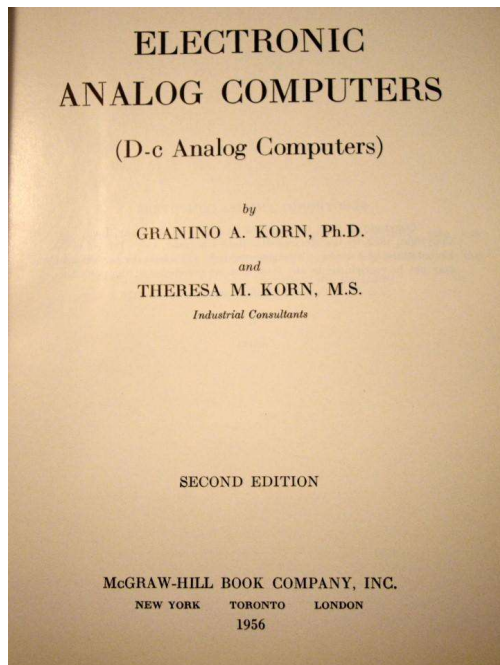
Kommandogerät für BMW 801 Flugmotoren

Mechanické systémy

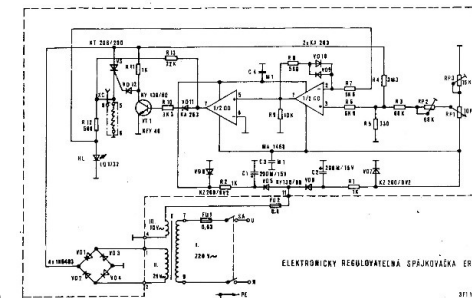
- Nevýhody:
 - přesnost a spolehlivost závisí na opotřebení – mění se s časem (dobou používání),
 - citlivost na prach, teplotu, otřesy,
 - vyžaduje častou údržbu,
 - náročný návrh – součásti se konstruuje přímo pro konkrétní účel, obtížné dodatečné změny.

Elektronické systémy

- Pokrok v technologiích (elektronika pevné fáze látek) umožňuje navrhovat a vyrábět systémy jako elektronický obvod.



... zpočátku spojitě
(analogové), což
mohou být v podstatě
elektronické obdoby
mechanických předchůdců



a později diskrétní (číslicové).

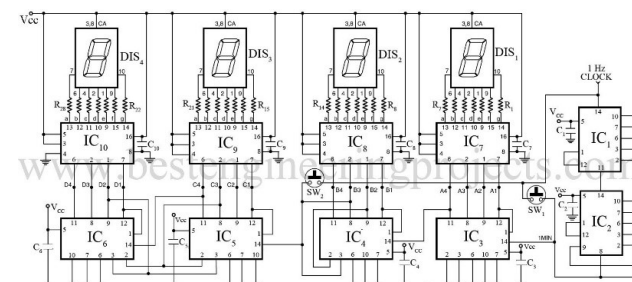
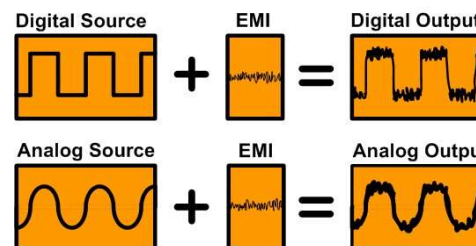


Figure 1: 24 Hour Digital Clock Circuit

Elektronické systémy

V čem je pokrok oproti mechanickým systémům?

- Odstraněno mechanické opotřebení!
- Menší a lehčí,
- flexibilnější, pokud jde o zdroj a přívod energie, snímání vstupních veličin (elektrická vazba je lepší než mechanická) a realizaci výstupů (elektrické signály se pak převádí na silové/energetické působení téměř libovolného rozsahu, mechanický výstup obvykle „téměř nic neutáhne“ a ještě má při přenosu ztráty).
- Jednodušší návrh – součásti se vyrábí průmyslově (kvalita), návrhář je prostě koupí a navrhne pouze finální sestavu.
- Číslicový obvod má navíc jasně definovanou přesnost, na kterou nemá vliv stárnutí součástek, okolní šum, kolísání napájení atd.



Dnes: systém jako software

V čem je pokrok oproti elektronickým systémům?

- Snadný a komfortní návrh – popíšeme chování systému (v nějakém vhodném jazyce) – a to je celé! **Vytvářením popisu vlastně rovnou vzniká sám systém.**

Komfort - umožňuje navrhovat složitější systémy,
- umožňuje navrhovat levněji,
- otevírá možnosti více lidem realizovat své nápady,

- Vysoká flexibilita – systém lze snadno změnit, doplnit, rozšířit, opravit.

Příklad: chci postavit digitální hodinky

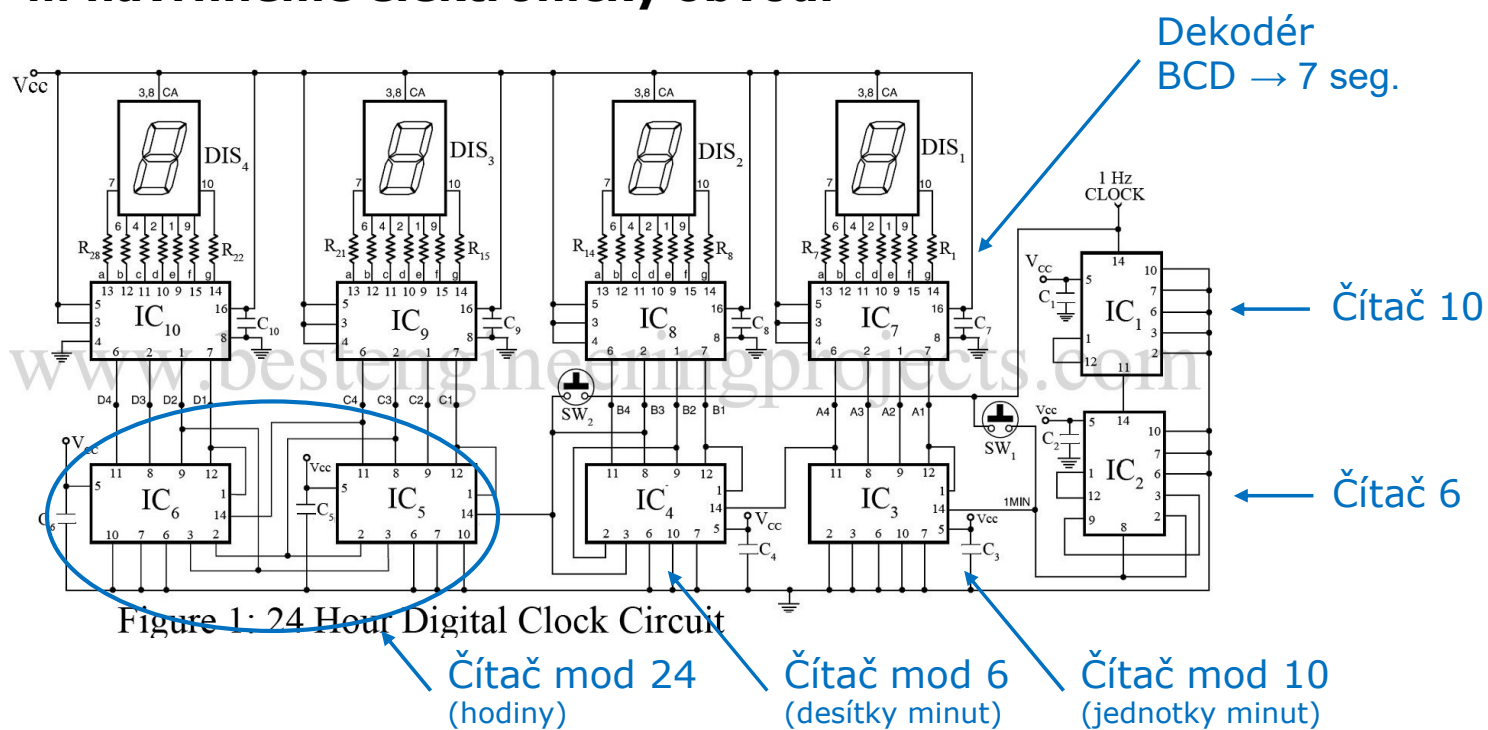
= elektronický přístroj, který má měřit a zobrazovat čas.



☞ Porovnejme způsob realizace čistě hardwarové a softwarové.

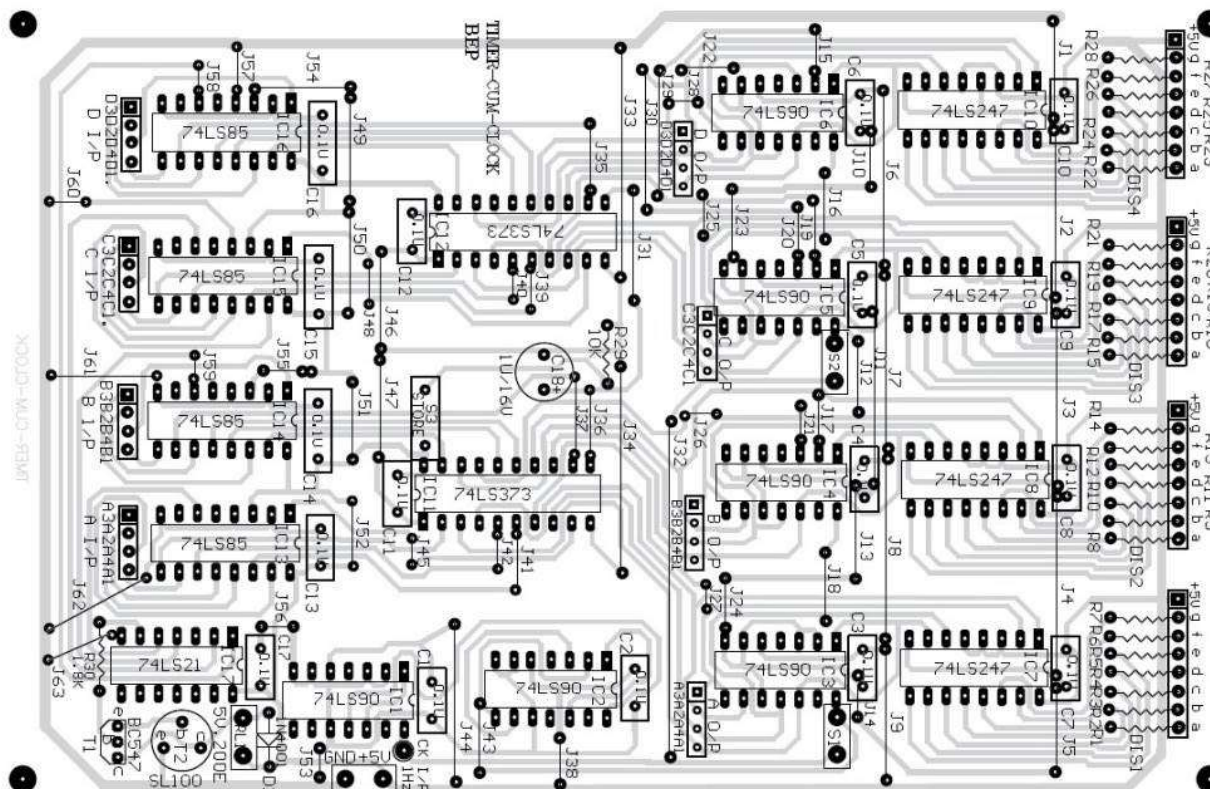
Jak na to?

... navrhne elektrický obvod.



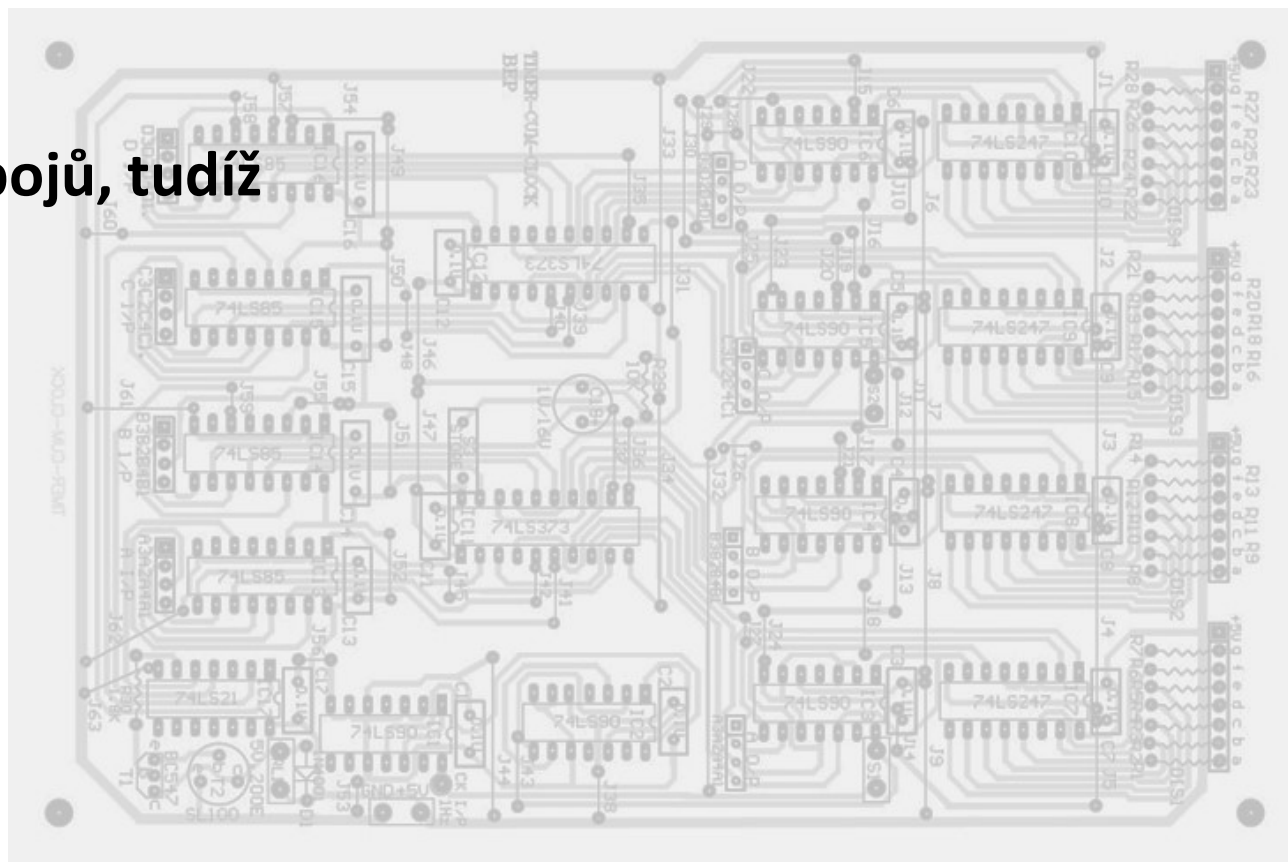
Digitální hodiny jako elektronický obvod

Fyzická realizace na desce plošných spojů:



Digitální hodiny jako elektronický obvod

- **Pěkné, ale:**
 - mnoho součástek a spojů, tudíž
 - drahé,
 - rozměrné a těžké,
 - nespolehlivé.



... leda by se to dalo všechno nějak miniaturizovat a dát na jeden čip.

System On Chip (SOC)



Náramkové hodinky
Hamilton Pulsar z roku 1970.

Cena \$2100 (přepočteno na dnešní
hodnotu dolaru \$12000)

V roce 1973 je nosil James Bond
ve filmu Žít a nechat zemřít.



Digitální hodiny jako čip (SOC)

- **Pěkné, ale:**

- **drahé na návrh a výrobu** (velké fixní náklady - jednoúčelový čip)

- aby se vyplatilo, je třeba to dát na ruku panu Bondovi a pak chrlit ve velkých sériích

- **málo flexibilní**

- představte si, že někdo přijde a řekne, že to máte předělat na 12-hod. cyklus namísto 24-hod cyklu...

Nebylo by lepší to udělat jako software?

Digitální hodinky jako software

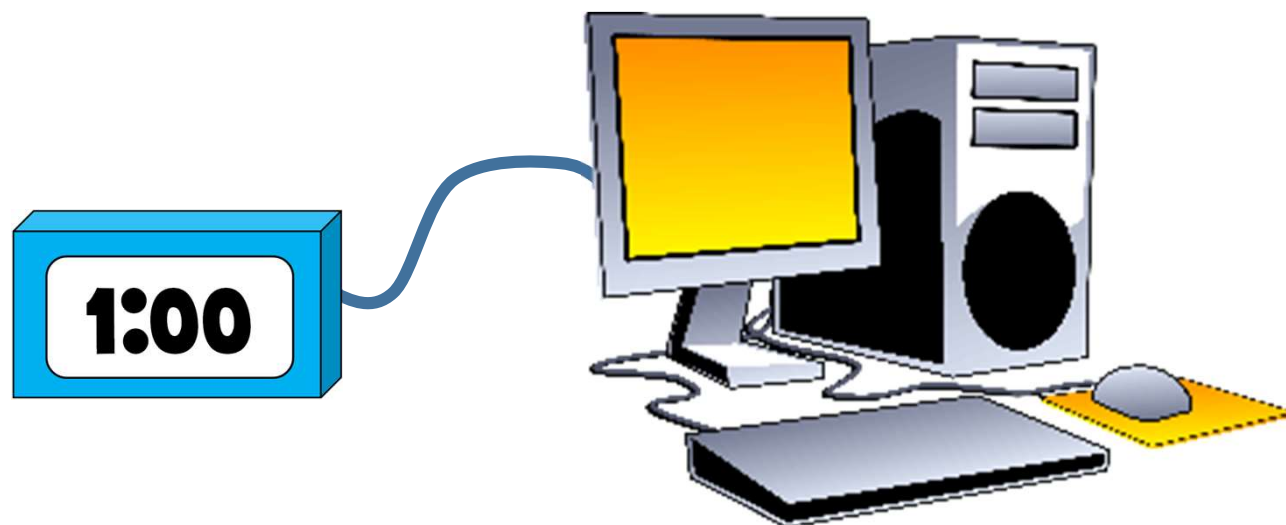
- Hodinky jako software?
- Proč ne, popsat jejich chování v nějakém programovacím jazyce bychom určitě dokázali:

```
if (milliseconds >= 1000) {  
    seconds++;  
    milliseconds=0;  
}  
  
if (seconds >= 60) {  
    minutes++;  
    seconds = 0;  
}  
  
if (minutes >= 60) {  
    hours++;  
    minutes = 0;  
}  
  
if (hours >= 24) {  
    hours = 0;  
}
```

```
#define N0 0b00111111  
#define N1 0b00000110  
#define N2 0b01011011  
#define N3 0b01001111  
#define N4 0b01100110  
#define N5 0b01101101  
#define N6 0b01111101  
#define N7 0b00000111  
#define N8 0b01111111  
#define N9 0b01101111  
  
void sn(int number, byte display) {  
    byte n;  
  
    switch (number) {  
        case 0:  
            n = N0; break;  
        case 1:  
            n = N1; break;  
        case 2:  
            n = N2; break;  
        case 3:
```

Digitální hodinky jako software

OK, program napíšeme, ale na čem poběží, aby to nebylo nakonec horší než „hardwarové“ řešení?



Proč vestavěné systémy?

- Software je nehmotný, nemůže fungovat sám o sobě, musí běžet na počítači.
- Řada systémů **může být provedena jako software v počítači** (aplikace),

ale hodně systémů potřebujeme mít fyzicky. Pouze chceme definovat jejich chování pomocí software.

To vede na **elektronické obvody, do nichž je**, pro účely definování chování (řízení) vestavěn počítač.

Sám počítač je vlastně také (elektronický) systém.
Proto hovoříme o vestavných systémech.

Počítač jako vestavný systém

Jak vypadá a čím se liší od jiných počítačů?

A co má se všemi ostatními počítači naopak společné?

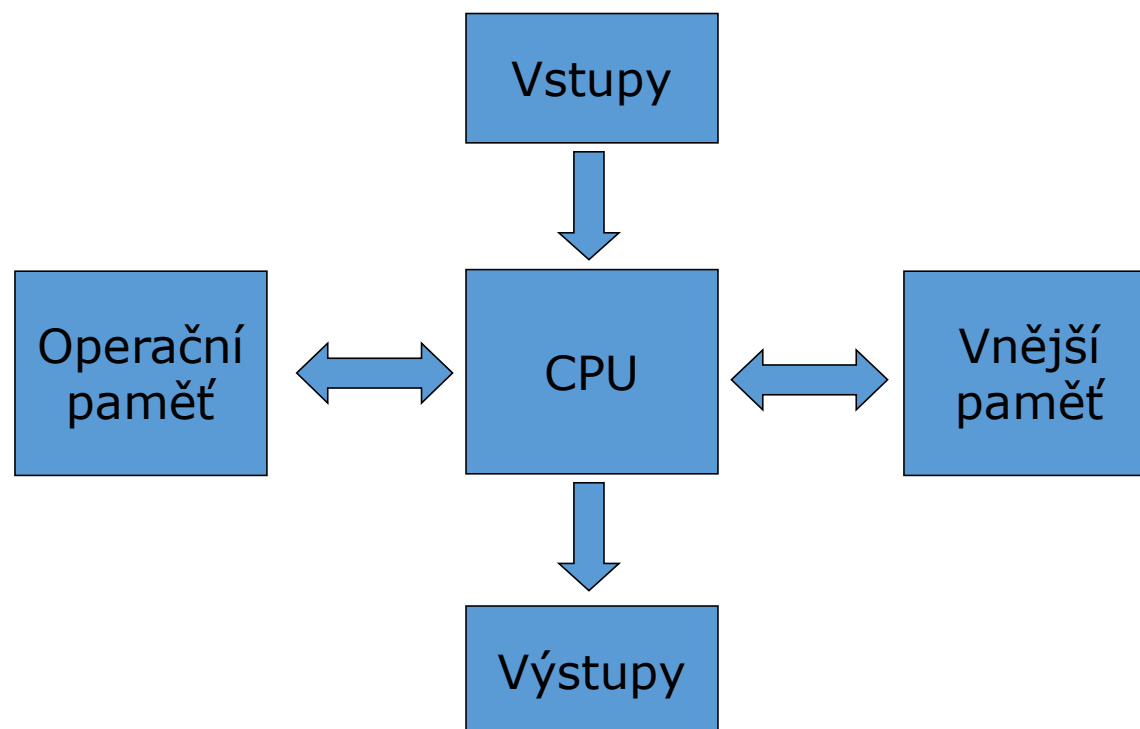
Vestavný počítačový systém OBECNĚ

- Máme problém, který bychom **chtěli/potřebovali** **řešit** (alespoň částečně) **softwarově**.
- **Aby software běžel, potřebujeme počítač(ový systém).**
- Ale nechceme tam mít celý běžný počítač (PC).
- **Navrhneme vestavný počítačový systém** (součástí toho systému bude i software, řešící náš problém).

Co to je počítač?

- Nejobecnější schéma číslicového počítače

(von Neumannovo schéma) – co potřebujeme, aby běžel program?



Jaký počítač?

- Potřebujeme **určitě procesor a** (operační) **paměť**, aby náš program „někdo“ vykonal.
- **Procesor** obvykle **nemusí být moc rychlý** (nechceme třeba přehrávat HD video, ale jen počítat čas) a **paměť nemusí být moc velká**.
- Jako **vstupy a výstupy nepotřebujeme GUI** a k tomu monitor, klávesnici, myš. Stačí jen **umět se připojit na okolní elektrické obvody** – **vestavět počítač do aplikace**.

Kde vzít takový počítač?

- Ukazuje se, že **výše uvedené požadavky** na počítač **se v průmyslu objevují „poměrně často“** (desítky miliard kusů ročně).
- Proto existuje velmi prosperující odvětví, které řešení dodává (a nejlevnější doslova za pár korun).

Mikrokontrolér (MCU)

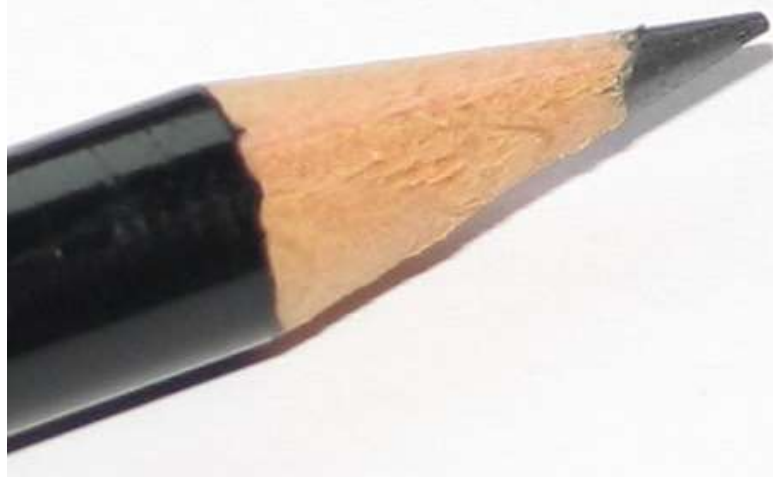
Malý levný počítač, který umí vykonávat program (např. napsaný v C) a podle něj **řídit zařízení** (elektrické obvody), **do kterého je vestavěn**, se nazývá

MIKROKONTROLÉR

Jak vypadají?

Například:

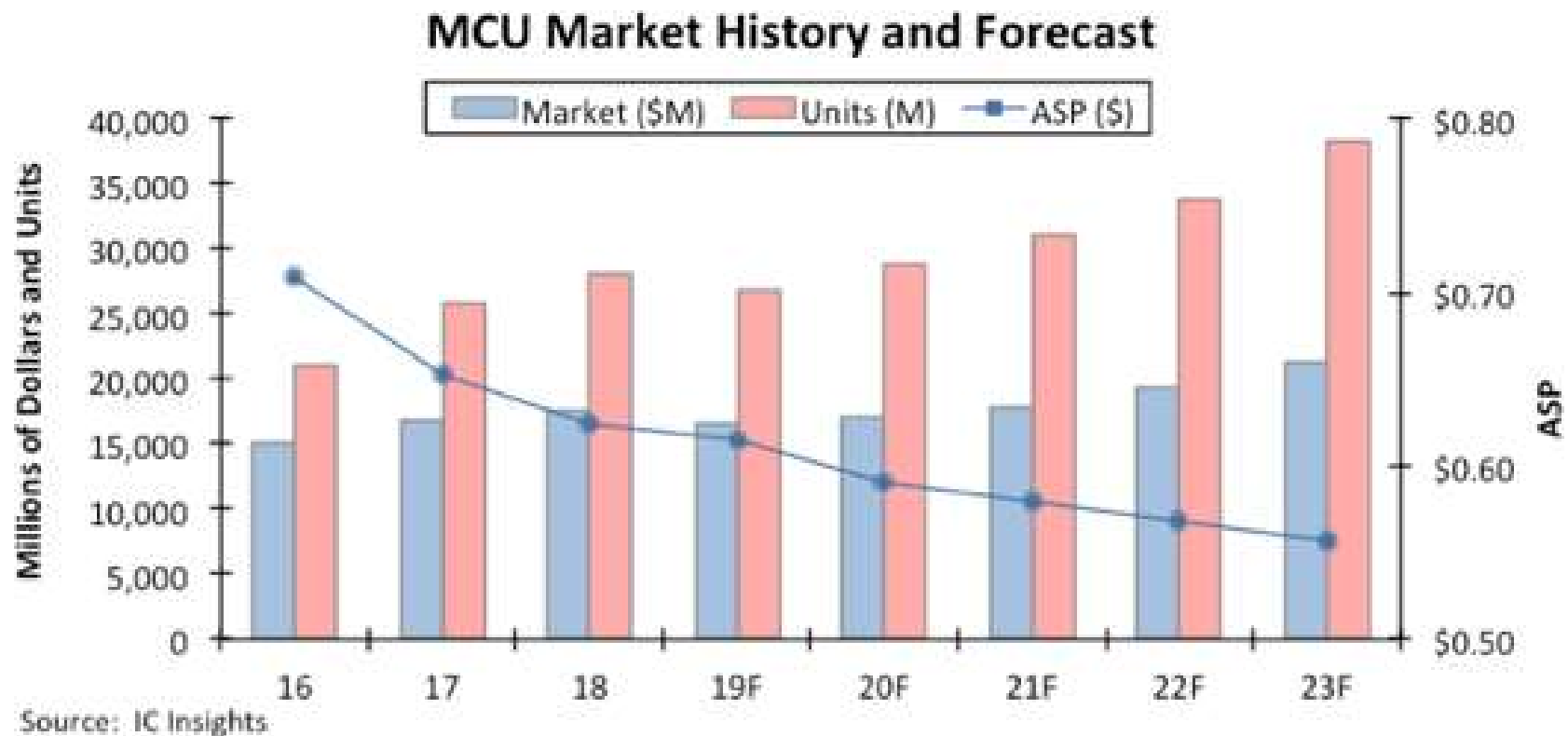
NXP (Freescale)
S08QE8



Všechny obvody počítače jsou ukryté v plastovém pouzdře

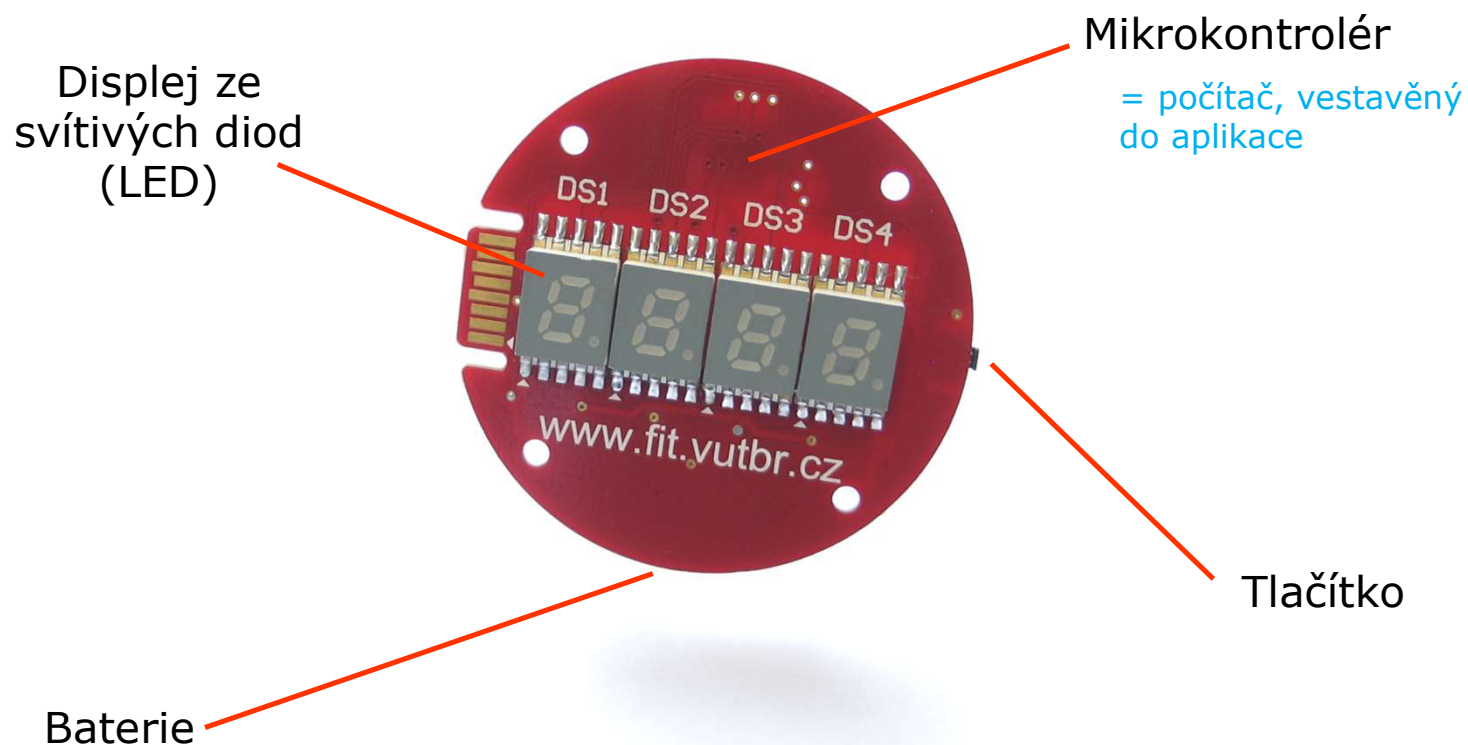
Kovové vývody slouží k připojení vstupů a výstupů

Prodej mikrokontrolérů na světovém trhu

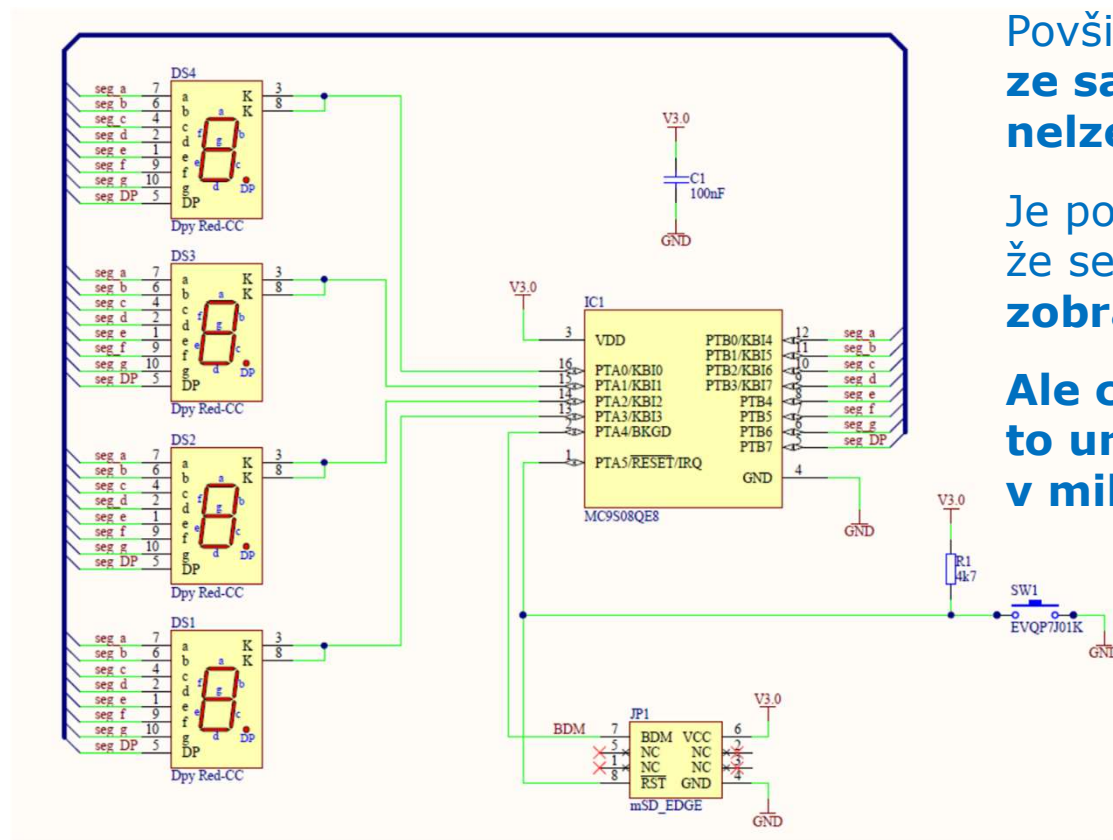


> Tento předmět se zabývá věcmi, kterých se ročně prodá desítky miliard kusů!

Hodinky s mikrokontrolérem



Hodinky s mikrokontrolérem



Povšimněte si, že
**ze samotného schématu
nelze zjistit funkci.**

Je pouze zřejmé,
že se bude **něco
zobrazovat na displeji.**

**Ale co a jak,
to určuje software
v mikrokontroléru.**

... jestli to budou hodinky nebo třeba počítadlo předmětů, určí jen software, který do mikrokontroléru nahrajeme a na něm spustíme.

Pozor!

- Příklad *nesloužil* k tomu, abychom demonstrovali, že „softwarové“ řešení je lepší než „hardwarové.“

To nemusí být úplně vždy pravda, ale
je celá řada situací, kdy se to hodí – a to
jsou příležitosti pro nasazení vestavných
systémů

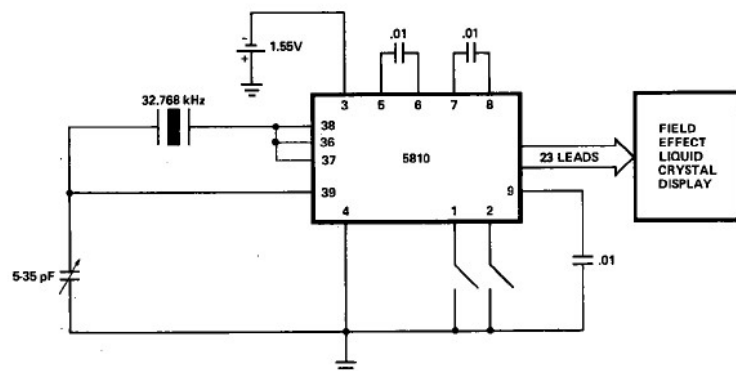
➤ Najdete nějaké přednosti hardwarového řešení podobných problémů?

Například spotřeba

Intel 5810A

(Single chip LCD time/second/date watch circuit)

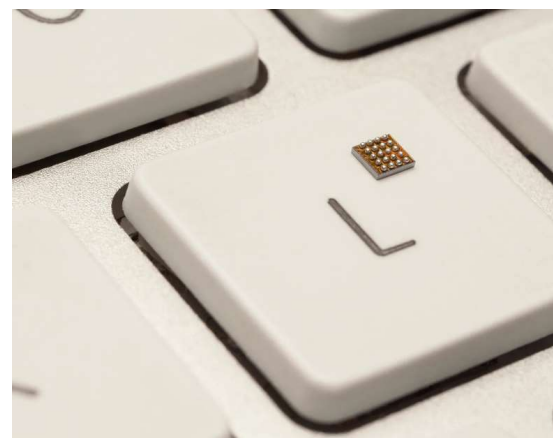
Typical Application



Vyráběn od roku 1974, spotřeba 5 μ A.

NXP Kinetis KL02

(ARM Cortex M0+ based microcontroller)



Vyráběn od roku 2013,
spotřeba při úsporném běhu 185 μ A.

Vestavné systémy

(Embedded Systems)

nějakou představu už máme, teď trochu obecněji

Co je vestavný systém (Embedded System)?

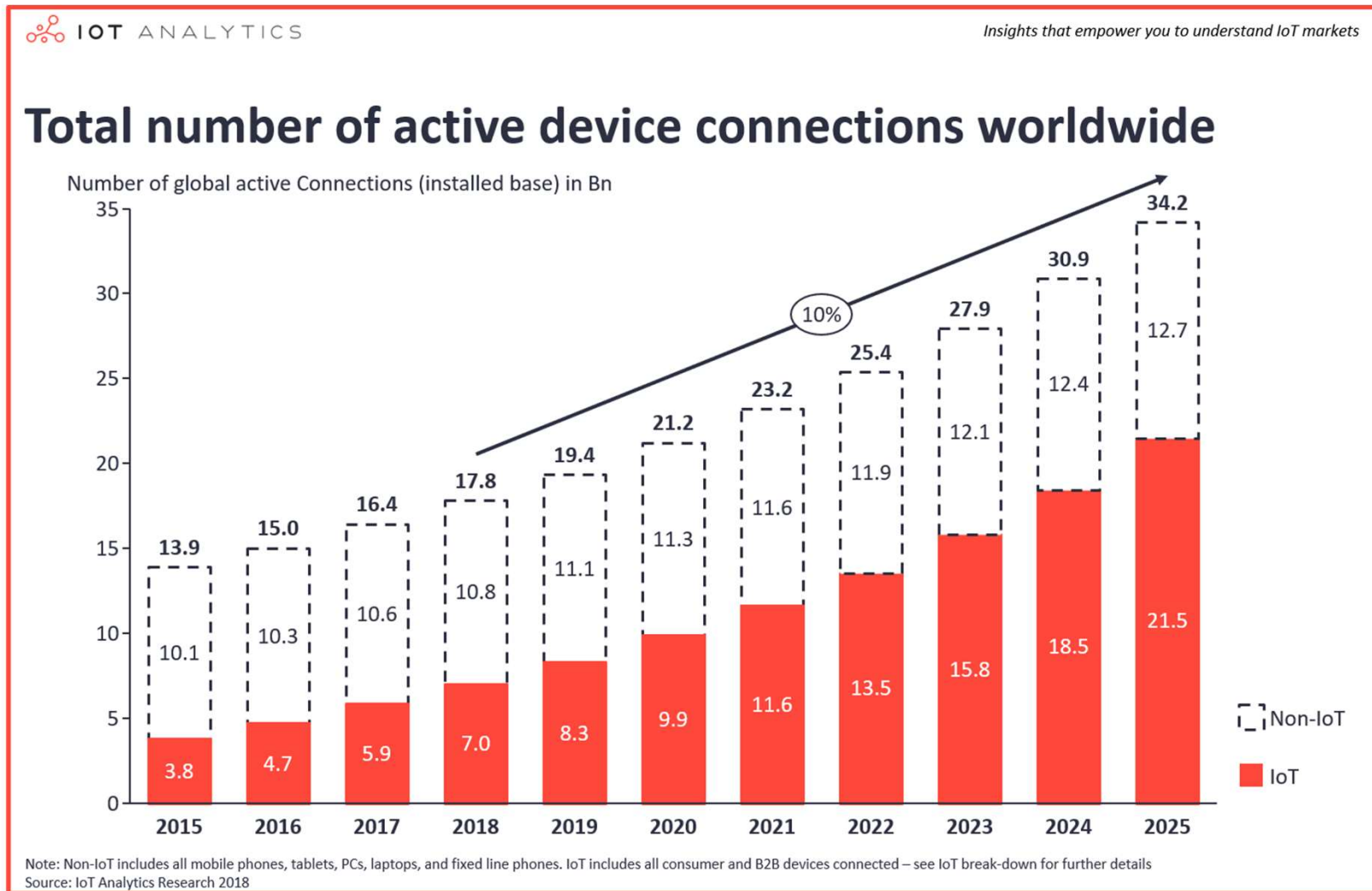
- Dokument řídicího výboru ESTP (Embedded System Technology Platform) Evropské strategické iniciativy definuje embedded systems jako **kombinaci hardwaru a softwaru, jejímž smyslem je řídit externí proces, zařízení nebo systém.**
- Počítač, který je zabudován do systému, ale pro uživatele není jako počítač viditelný.
- Slouží pro řízení celého systému a poskytuje výpočetní podporu pro jednotlivé komponenty systému

Výběr aplikací vestavných systémů

- Automobily a další stroje
- Komunikační a měřicí přístroje
- Veškeré domácí spotřebiče
- Počítačové komponenty
a periferie
- Průmyslové aplikace
(Průmysl 4.0)
- Robotika a autonomní systémy
- Senzorové sítě



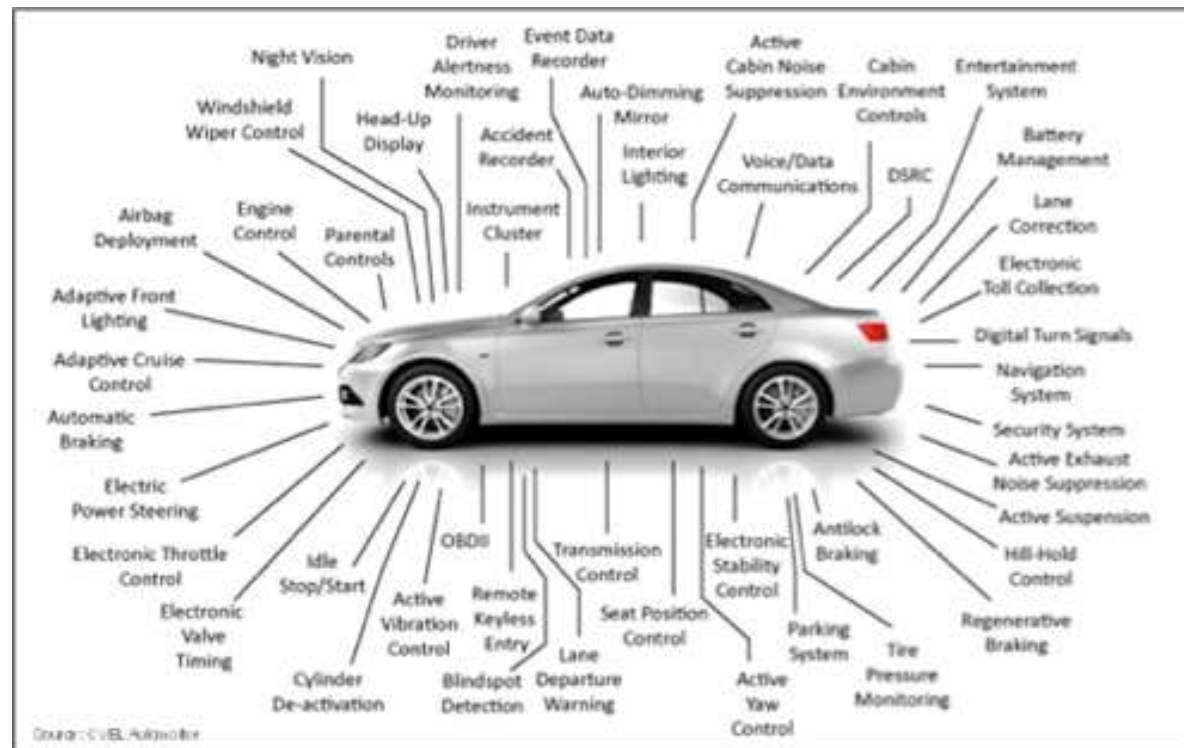
Internet of Things



Vestavěné systémy v automobilech

Početné procesory propojené sítí (~ 100), s variétou typů:

- 8-bit. – uzamykání dveří, světla, stěrače, atd;
- 16-bit. – většina funkcí; 32-bit – řízení motoru, airbagů,..
- Více jak 30% ceny auta tvoří elektronika



Vestavěný systém x univerzální počítač

- Jeden program po celý život, specifický pro aplikaci.
- Uživatel by neměl ani tušit, že pracuje s počítačem.
- Hlavní interakce nemusí být s člověkem – čidla pro snímání prostředí, ovládání akčních členů.
- Startuje sám bez lidského zásahu.
- Uživatel spouští nejrůznější programy, jaké zrovna potřebuje.
- Periferie hlavně pro interakci s uživatelem, který zadává vstupy a získává řešení.
- Nutnost operačního systému, soubory, ukládání dat,...

Je telefon vestavný systém?



Přístroj na telefonování
řízený počítačem

x



Počítač, který umožňuje
také telefonování

Realizace vestavných systémů

Mikrokontrolér není jediná možnost:

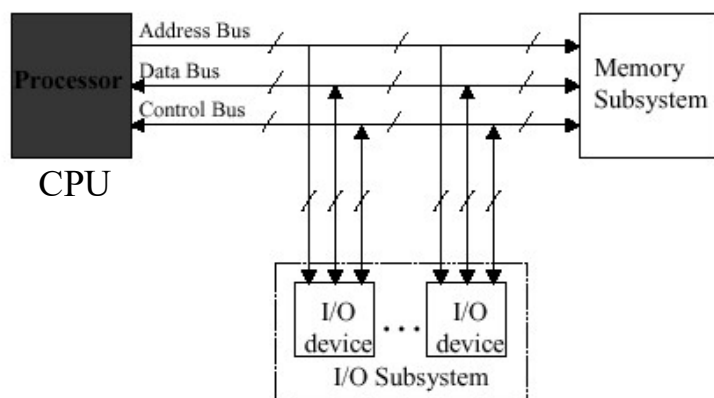
	Implementation	Design Cost	Unit Cost	Upgrades & Bug Fixes	Size	Weight	Power	System Speed
Dedicated Hardware	Discrete Logic	low	mid	hard	large	high	?	very fast
	ASIC	high (\$500K/mask set)	very low	hard	tiny - 1 die	very low	low	extremely fast
	Programmable logic – FPGA, PLD	low to mid	mid	easy	small	low	medium to high	very fast
Software Running on Generic Hardware	Microprocessor + memory + peripherals	low to mid	mid	easy	small to med.	low to moderate	medium	moderate
	Microcontroller (int. memory & peripherals)	low	mid to low	easy	small	low	medium	slow to moderate
	Embedded PC	low	high	easy	medium	moderate to high	medium to high	fast

Počítače pro vestavěné systémy

- Lze postavit klasický počítačový systém s CPU, pamětí,
- koupit modul,
- lze osadit mikrokontrolér (MCU),
- použít „soft-core“ např. do FPGA, pro ASIC, ...

Klasický počítačový systém

- Pro vestavěný systém se dnes už moc nevidí:
 - k dispozici je nepřeberné množství MCU různých výkonů,
 - a naproti tomu klasických CPU je omezený výběr, optimalizace pro klasické „desktop“ a „mobile“ aplikace.



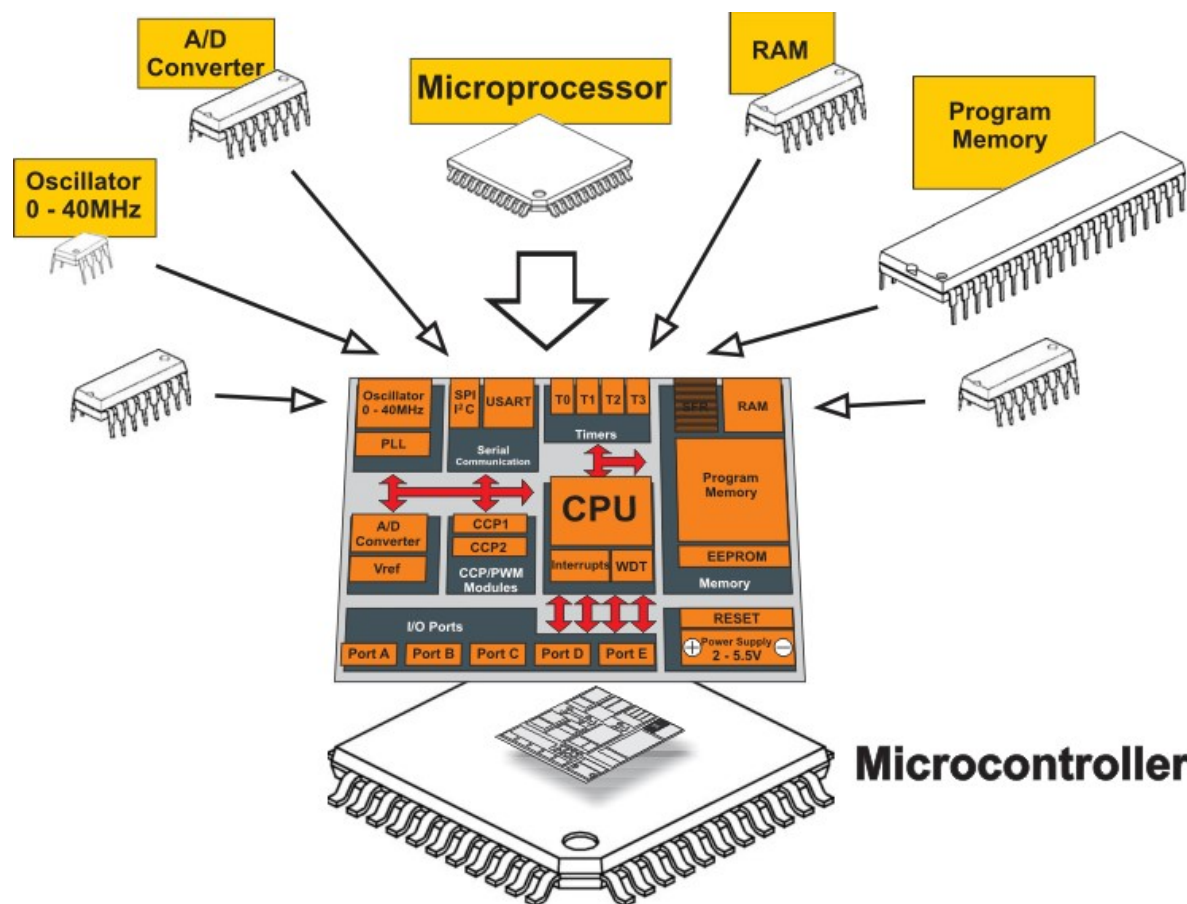
Moduly pro vestavné systémy

Typicky pro aplikace, kde se požaduje výkon a velká paměť (zpracování multimediálních dat, analýza signálů).



Mikrokontrolér (MCU)

= jednočipový počítač, optimalizovaný pro vestavění do aplikace

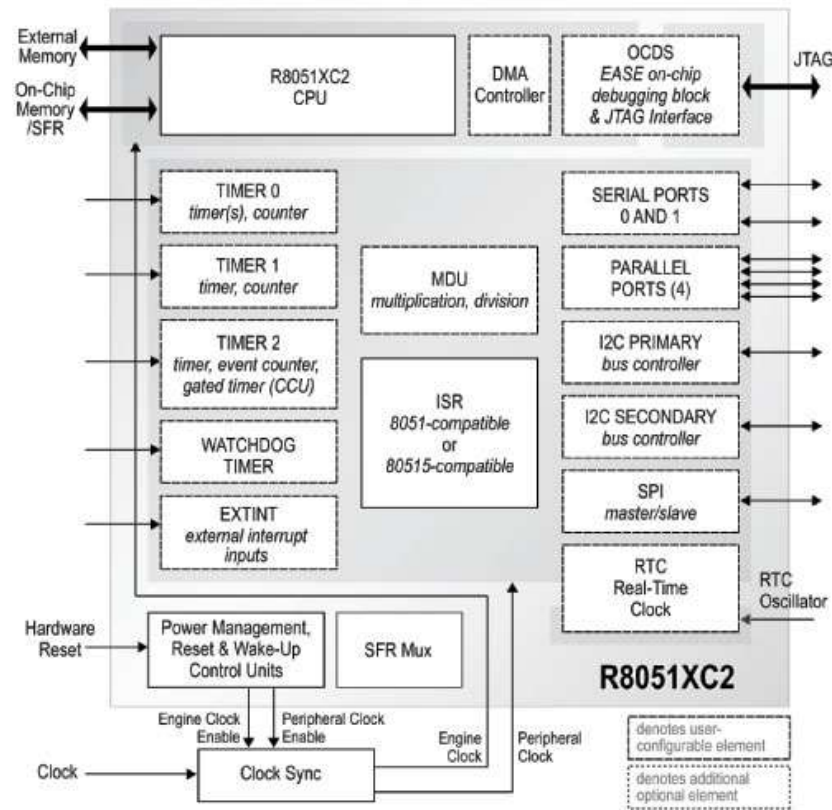


Soft Core MCU

- Příklad: CAST, inc.

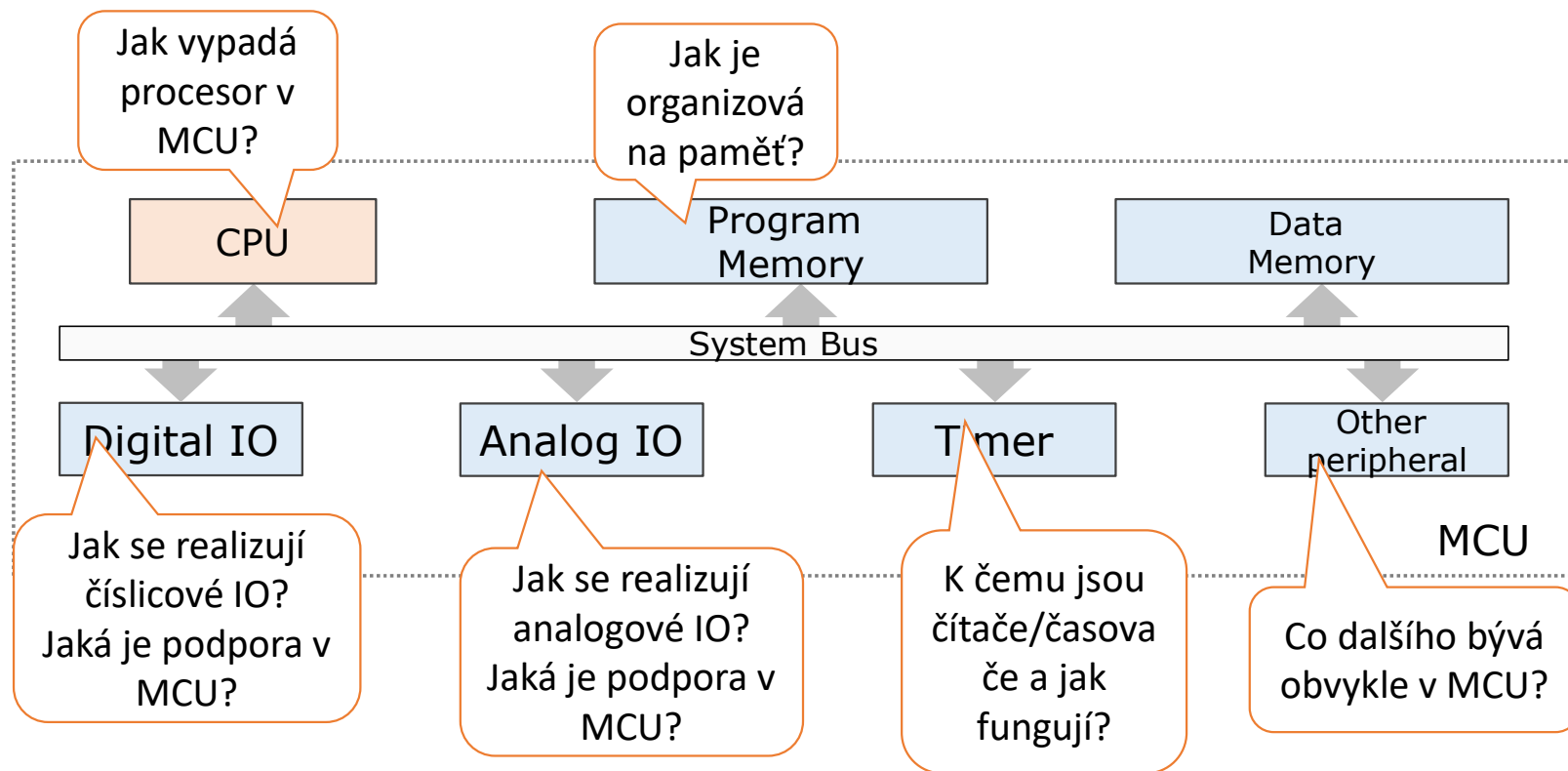
IP jádro 8051IP

- od 2 200 hradel
- od 21 μ W/MHz
- až 0,5 GHz
- 64k – 16M paměti



Co obvykle najdeme v mikrokontroléru?

... a o čem bude tento předmět.



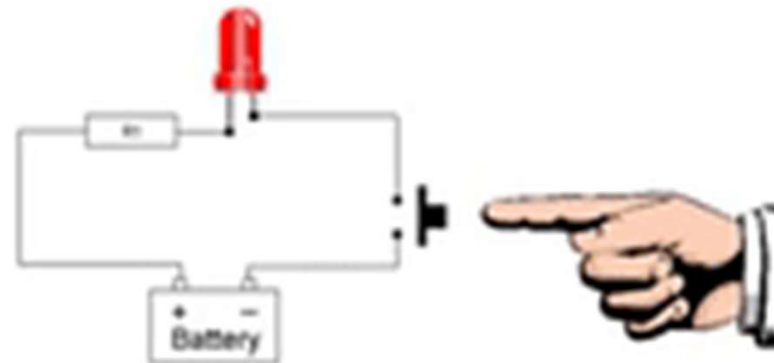
Jak chápeme roli mikrokontroléru?

- Mikrokontrolér (MCU) je elektronická součástka, kterou lze vložit do (prakticky libovolného) zařízení jako „**řadič**“/“**mozek**“ – elektronický obvod, přijímající a vydávající signály, kterými se řídí chování tohoto zařízení.
- Chování mikrokontroléru lze běžným způsobem naprogramovat – je to přece počítač.

Naprogramovat mikrokontrolér = popsat chování zařízení pomocí software!

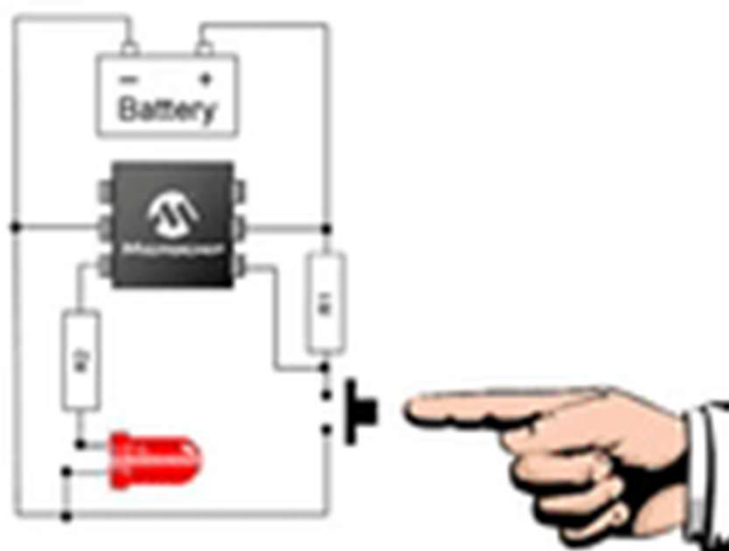
Where would I use a Microcontroller?

- Anywhere you would like to add intelligence...



Where would I use a Microcontroller?

- Anywhere you would like to add intelligence...

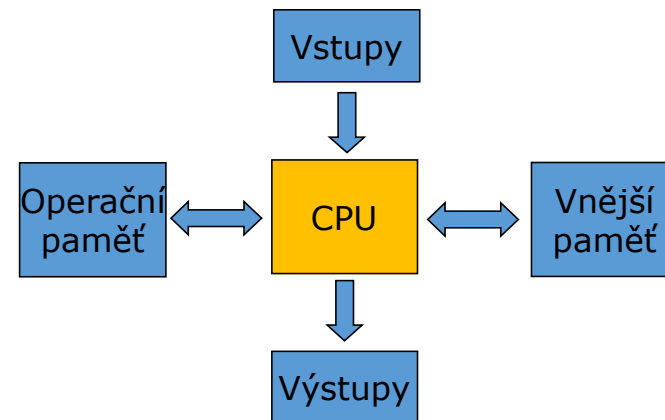


Požadavky na moderní mikrokontrolér

- Nízká cena.
- Nízká spotřeba, možnost ovlivnit spotřebu.
- Relativně výkonný procesor.
- Optimalizace pro programování ve vyšším jazyce (typicky C).
- Podpora ladění aplikací, snadný přístup „dovnitř“ pro vývojáře.
- Komunikační rozhraní pro rozšiřitelnost, bezdrátová komunikace.
- Flexibilita vstupů a výstupů pro připojení co nejširší škály vnějších prvků.
- Omezení rušení, elektromagnetická kompatibilita.
- Bezpečnostní prvky, ochrana.

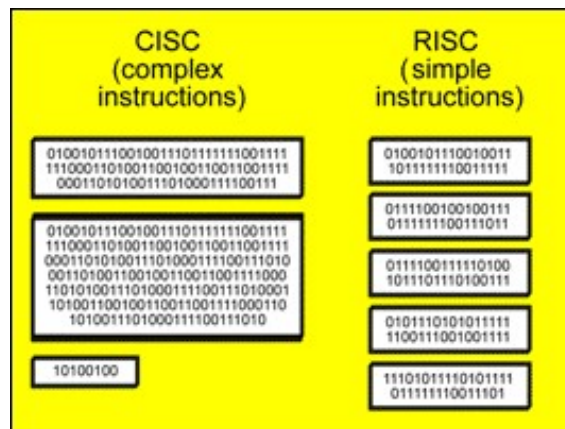
Počítač v mikrokontroléru,

jádro = procesor počítače v mikrokontroléru

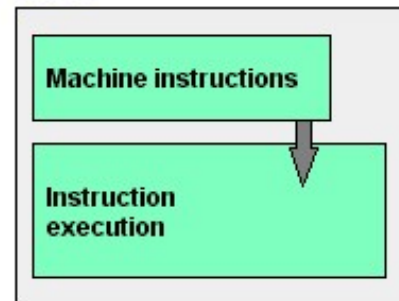


CISC x RISC

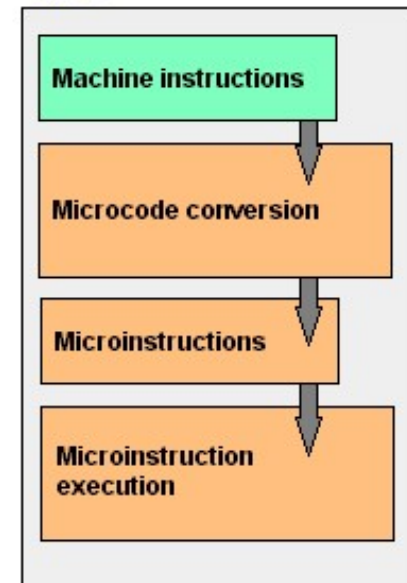
CISC – Complex Instruction Set Computer
RISC – Reduced Instruction Set Computer



RISC



CISC



CISC x RISC

CISC

- Mnoho typů instrukcí s mnoha variantami a mnoha adresovacími režimy.
- Typicky memory-to-memory instrukce (výběr operandů a uložení výsledků je součástí instrukce).
- Proto nebývá mnoho datových registrů, zpravidla jeden hlavní „střadač“ (Accumulator).
- Instrukce trvají více taktů.
- Na čipu dominuje logika pro implementaci instrukcí.

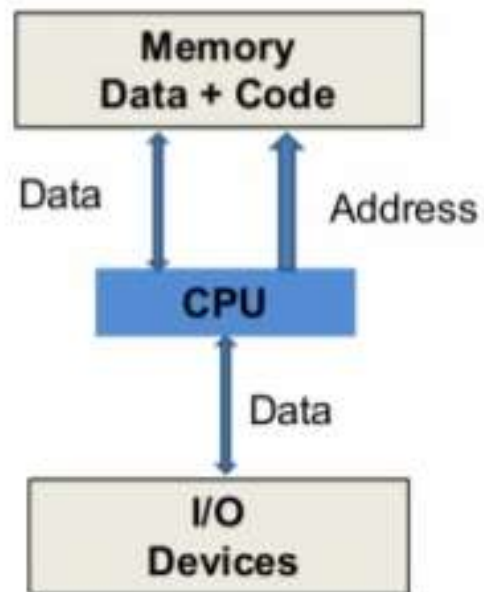
RISC

- Několik základních instrukcí, jednoduché režimy adresování.
- Typické jsou instrukce register-to-register, proto jsou separátní instrukce „LOAD“ a „STORE.“
- Typicky mnoho datových registrů.
- Instrukce trvají pokud možno všechny stejně, ideálně jeden takt.
- Na čipu dominuje paměť – registry.

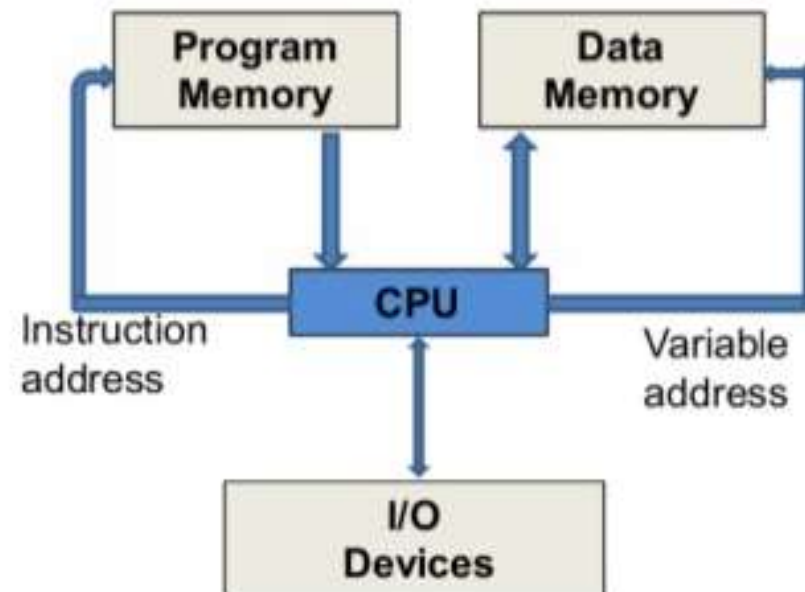
CISC x RISC

- Dnešní rychlá paměť (nevadí hodně přístupů) a dobré kompilátory (i z vyššího jazyka lze získat efektivní kód) činí řadu výhod CISC architektur spornými.
- Původní hranice se smazávají, procesory „umí“ hodně instrukcí a přitom je umí provádět rychle a efektivně.

Von Neumannská x Harvardská architektura



Von Neumann Machine



Harvard Machine

Von Neumannovská x Harvardská architektura

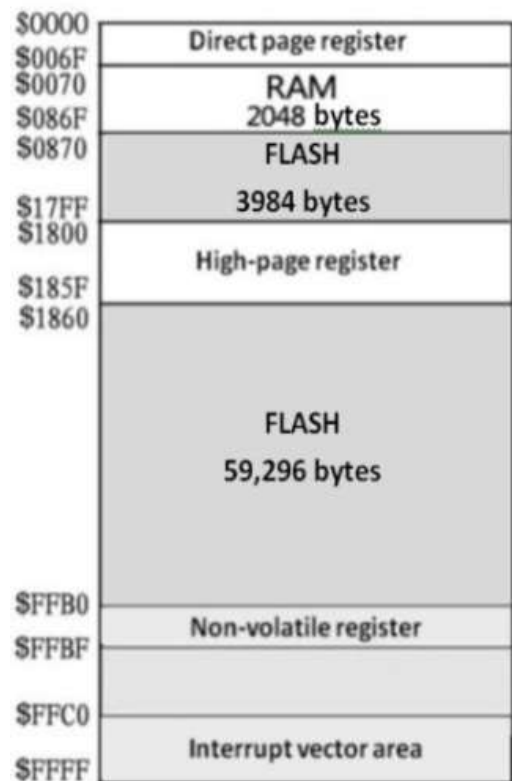
- Jediná operační paměť, jediný adresový prostor.
 - Flexibilnější pro (měnící se) aplikace.
 - Možnost samomodifikujícího se kódu.
 - Sběrnice je úzké hrdlo (instrukce a operandy k nim musíme z paměti vybírat postupně).
- Instrukce a data mají každá „svoji“ paměť – dvojí adresový prostor!
 - Umožňuje použít fyzicky rozdílné technologie paměti – např. pro program FLASH a pro data RAM.
 - Umožňuje použít rozdílnou velikost buňky paměti – např. 8 bitů pro data a 18 bitů pro instrukci.
 - Umožňuje v jednom taktu získat instrukci a zároveň operandy pro ni.

Von Neumannovská x Harvardská architektura

- U univerzálních počítačů zatím vítězí „von Neumann“ díky flexibilitě paměťového prostoru.
 - U vestavných systémů (mikrokontrolérů) se používají oba přístupy.
- **Program** zde bývá **typicky trvalý** (reprezentuje popis chování systému, ten je daný), ukládá se do paměti typu FLASH, **data** jsou **typicky proměnná** (reflektují momentální stav systému, který se mění, jak se mění podmínky, sledované okolí, režim činnosti atd.)

Příklad von Neumannovského MCU

- Rodina MCU Freescale/NXP S08/S12



Jediný adresový prostor obsahuje paměť programu typu FLASH i paměť dat typu RAM.

Výhody:

- Konstanty (tabulku konstant) lze mít klidně ve FLASH, přistupuje se k nim běžně jako k jakýmkoli jiným datům (to je u Harvardské architektury trochu problém).
- Program může běžet i v RAM jednak pro možnost samomodifikace, ale hlavně se používá pro low-power aplikace, protože RAM „žere“ méně než FLASH (kritický úsek programu zkopírujeme do RAM, tam spustíme a FLASH můžeme vypnout).

Příklad Harvardského MCU

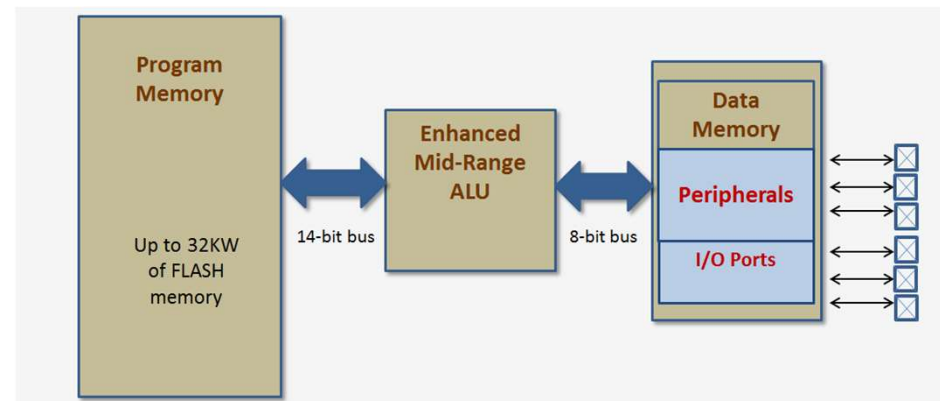
- Rodina Microchip PIC

Paměť programu (FLASH):

- Jedna adresa = vždy jedna celá instrukce (osvobození od bajtového zarovnání) = **snadné a rychlé provádění instrukcí, typický RISC** se vším všudy.

Paměť dat (RAM):

- V podstatě **pole univerzálních a účelových registrů** = krátká adresa může být zakódována v instrukcích jako operand = netřeba přidávat LOAD/STORE instrukce.



Mikrokontrolér není „jen počítač“!

je toho v něm mnohem víc...

Proč MCU není *jen* počítač?

- **Počítač v MCU určitě je**, je ústřední a nejdůležitější komponentou.

... ale není to vše, ještě jsou tam další věci (a v tomto předmětu si budeme povídat hodně i o těch „dalších věcech“ v MCU, protože počítače už dobře znáte)

- Jsou tam **obvody pro vstupy a výstupy** (protože počítač – má-li něco řídit – potřebuje vědět, co se okolo děje a nějak to ovlivňovat).
- A ještě to nestačí – je tam spousta modulů, které **pomáhají počítači, aby se nemusel zabývat prkotinami** a stíhal to podstatné.

Počítač je sekvenční, ale svět okolo, do něhož je zrovna tento určen, je „paralelní“ – děje se hodně věcí současně! Moduly okolo jádra mu pomáhají to zvládnout.

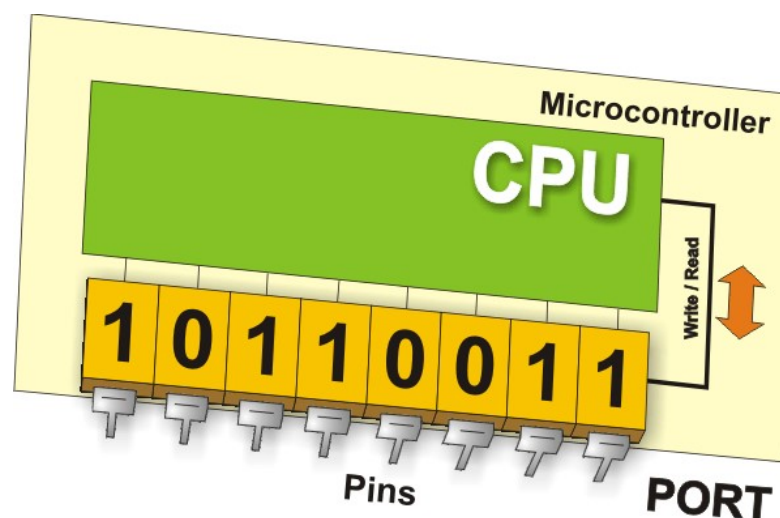
Vstupy a výstupy

- Jak zjistit stav okolního světa, aby na to mohl program reagovat?
 - Jak dostat informaci o momentálním stavu okolí do nějaké proměnné?
- Jak ovlivnit svět „venku“ z programu?
 - Jak podle výsledku výpočtu něco skutečně zapnout nebo vypnout?

Registry

- Protože počítač (procesor) **umí „jen“ přečíst obsah** (hodnotu) **z nějaké adresy nebo na ni nějakou hodnotu zapsat**, bude se vstup/výstup dělat také tímto způsobem.
- „Normálně“ jsou na adresách „jen“ paměťové buňky, které neumí víc než si pamatovat, co tam naposled bylo uloženo.
- Na určitých adresách může ale být něco, co uložením a zapamatováním dat nekončí:
 - zápis určité hodnoty něco způsobí v hardware „okolo“ nebo
 - hodnota se mění podle toho, co se „okolo“ děje.
- Říkáme jim „registry“, protože paměťová funkce tam je také přítomna
 - ale pozor neplést s registry v procesoru!

Příklad: jednoduchý vstup/výstup



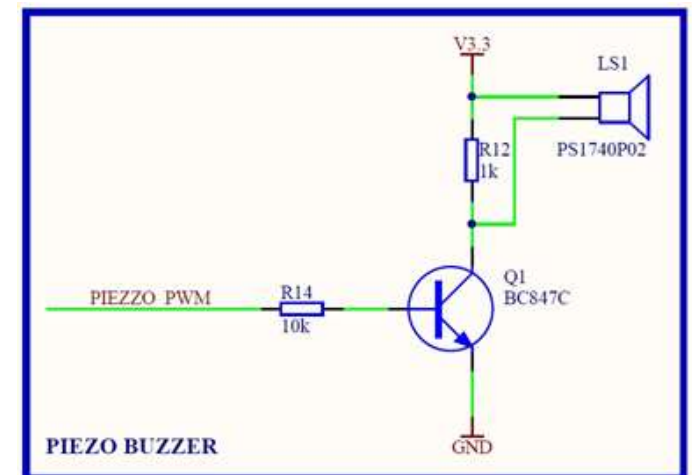
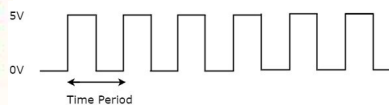
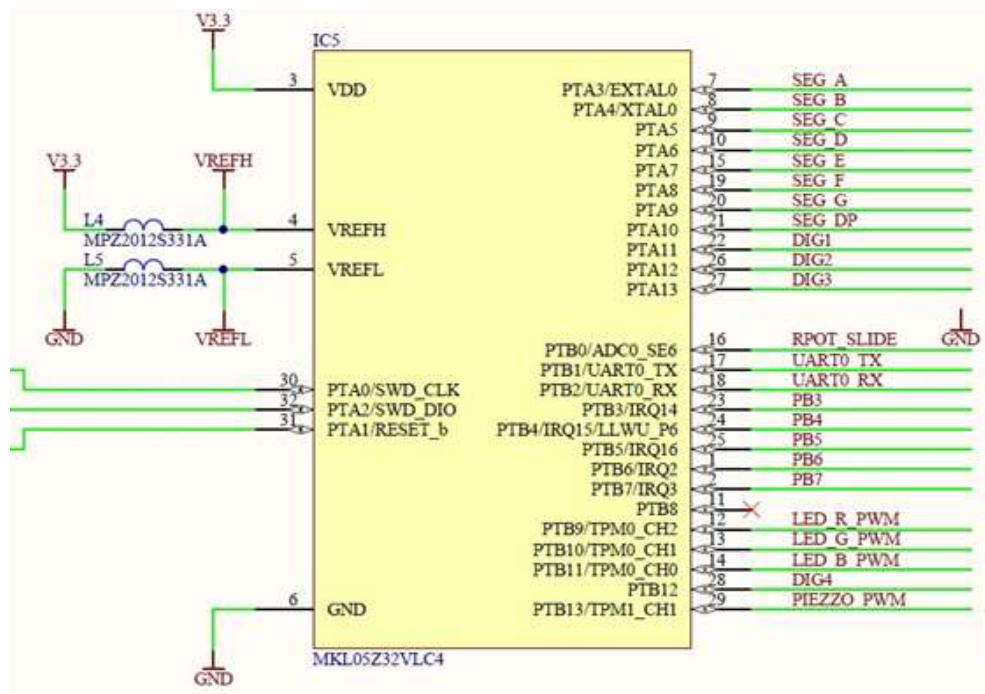
Výstup:

Zapíšu do registru 1 nebo 0 na příslušné místo → na vývodu se objeví 1 nebo 0.

Vstup:

Na vývod je přivedena 1 nebo 0 → hodnotu přečtu v registru na příslušném místě.

Příklad - pípnutí



Pípnutí - kód

```
for (;;) {  
    PTB->PDOR = GPIO_PDOR_PDO(0x2000);  
    delay(500);  
    PTB->PDOR = GPIO_PDOR_PDO(0x0000);  
    delay(500);  
}
```

Moduly v MCU

- Budeme se zabývat řadou dalších modulů, které se typicky vyskytují v mikrokontrolérech, ukážeme si na konkrétním příkladu MCU, jak fungují, jak se programují/nastavují a k čemu jsou dobré. Některé si zkusíte využít v laboratořích.
- Moduly v MCU typicky dělají činnosti, které je potřeba v aplikacích
 - dělat často,
 - jsou rutinní,
 - bylo by fajn je dělat „na pozadí“ (aby se procesor nemusel věnovat jen jim, ale mohl být k dispozici pro „důležitější“ věci).

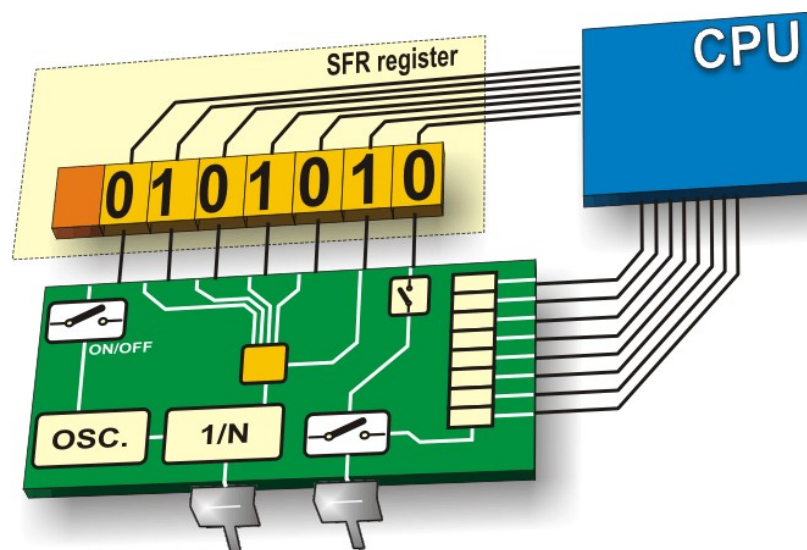
Takové činnosti se pak „zadají“ příslušnému modulu a dál se na ně může „zapomenout“ nebo se jen zkontroluje výsledek.

Moduly v MCU

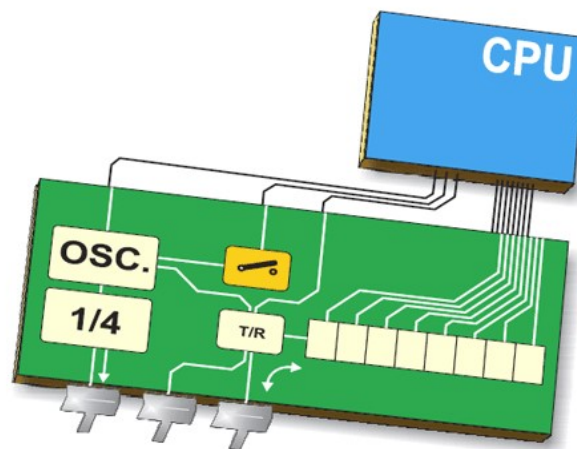
- Modul je typicky hardwarovou implementací nějaké funkce (kterou pak nemusíme programovat, ale jen si ji spustíme, když ji potřebujeme – výrobci MCU za ta desetiletí vývoje již dávno přišli na to, co „se hodí“).
 - Taková **implementace je efektivnější**, než kdyby to bylo naprogramováno jako sekvenční kód pro počítač – je to rychlejší, zabere to méně křemíku (plochy čipu), sežere to méně energie.
 - A hlavně: **běží to paralelně s počítačem**, což je blíže reálnému světu – postihnout děje reálného světa sekvenčním jednovláknovým počítačem naráží, jak sami víte, často na problémy.

Jak se moduly jeví programátorovi

- Na **definovaných adresách** jsou dostupné registry
 - do nich lze programem zapsat „co od modulu chci“
 - z nich lze přečíst momentální stav modulu či výsledek.



Příklad: sériová komunikace



Přijdou-li data zvenku, modul je přijme a jsou k dispozici v registru. Tam je program může přečíst. Jenže jak poznat, že data už přišla?

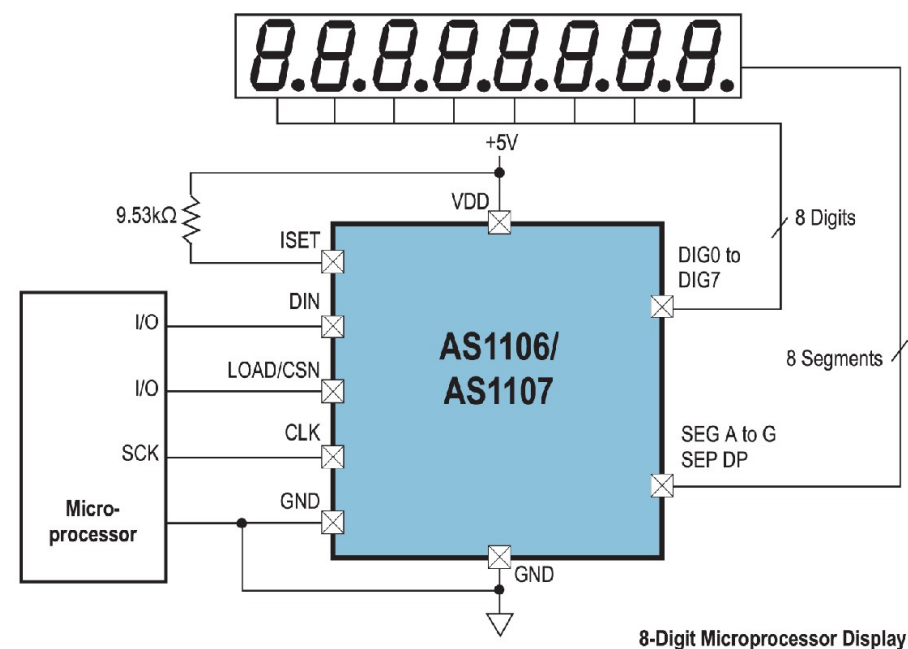
- musí existovat nějaký bit v nějakém dalším registru, který říká, zda máme nová data (příznak přijatých dat).

Tento příznak je třeba nejdříve otestovat a pak teprve číst data.

Nebo jinak – příchod dat způsobí přerušení a v jeho obsluze se čtou data.

Sériová komunikace ve vestavných systémech

- Nemusí sloužit jen ke komunikaci s vnějším světem.
- Velmi často se používá k realizaci jednoduché a efektivní sběrnice, pomocí které k mikrokontroléru snadno připojíme řadu pomocných obvodů, které v aplikaci zrovna potřebujeme.



Dokumentace!

- Dnešní MCU jsou složité – principy jsou vždy stejné (podobné) – ty si vysvětlíme na přednáškách a vyzkoušíme na cvičeních.
- Konkrétní věci ale musíme vždy hledat v dokumentaci.
- Typicky
 - Data Sheet ... MCU jako elektronická součástka
 - Reference Manual ... MCU z pohledu programátora – dokumentace k počítači na čipu
 - Application Notes ... příklady použití
 - Errata ... kde jsou známé chyby

Dokumentace k „našemu“ MCU

- Data Sheet: <https://www.nxp.com/docs/en/data-sheet/KL05P48M48SF1.pdf>
- Reference Manual:
<https://www.nxp.com/webapp/Download?colCode=KL05P48M48SF1RM> (po přihlášení)
nebo u nás:
https://moodle.vut.cz/pluginfile.php/907589/mod_label/intro/KL05P48M48SF1RM.pdf
- App. Note příklady: <https://www.nxp.com/docs/en/application-note/AN5075.pdf>
- <https://www.nxp.com/docs/en/application-note/AN4734.pdf>

Shrnutí

- MCU **můžeme brát jako čip se softwarem definovatelným chováním**,
- je to další krok od jednoúčelově navržených elektronických obvodů.
- **Přesto to není pouze počítač** (i když počítač tam dominuje), ale obsahuje další (jednoúčelově navržené) obvody, které se typicky stejně hodí.
- **Spojuje tak výhody softwarového** řešení (flexibilita, pohodlí a snadnost návrhu a ladění) **a hardwarového řešení** (efektivní implementace, rychlost, paralelismus).
- Ale pozor, jde o kompromis, který nemusí vyhovět vždy.
- Ukazuje se však, že **pro drtivou většinu aplikací je výborný**. Zbytek se musí řešit jinak.

O čem bude tento předmět

